



LangChain - Prompts

By 



LangChain - Prompts

Static Text prompts :

Text prompts are strings - ideal for straightforward generation tasks where you don't need to retain conversation history.

Ex: `response = model.invoke("Write a haiku about spring")`

- The prompt is fixed.
- No input variables.
- No reusability.
- Simple but not scalable.

Dynamic Text prompts - using `formatstring \ fstring`: This is just Python string formatting.

```
from langchain_openai import ChatOpenAI
```

```
model = ChatOpenAI(model = "gpt-4o")
```

```
topic = "machine learning"
```

```
prompt = f"Explain {topic} in simple terms."
```

```
response = model.invoke(prompt)
```

```
print(response.content)
```

Issues:

- **No prompt validation.**
- **Doesnot support the chains**



LangChain - PromptTemplate

Dynamic Prompts - Using PromptTemplate : This is the proper LangChain way.

```
from langchain_core.prompts import PromptTemplate
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

prompt_template = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple terms."
)

prompt = prompt_template.invoke({"topic": "machine learning"})

response = model.invoke(prompt)
print(response.content)
```

-- Using with chains

```
from langchain_core.prompts import PromptTemplate
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

prompt = PromptTemplate.from_template(
    "Explain {topic} in simple terms."
)

chain = prompt | model

response = chain.invoke({"topic": "machine learning"})
print(response.content)
```



LangChain – Messages

Messages are primarily used with **chat models**, where interactions are structured as **conversational** turns, rather than simple text generation tasks.

System message - Tells the model how to behave and provide context for interactions

Human message - Represents user input and interactions with the model

AI message - Responses generated by the model, including text content, tool calls, and metadata

```
messages = [  
    {"role": "system", "content": "You are a poetry expert"},  
    {"role": "user", "content": "Write a haiku about spring"},  
    {"role": "assistant", "content": "Cherry blossoms bloom..."}  
]  
response = model.invoke(messages)
```

```
from langchain_google_genai import ChatGoogleGenerativeAI  
from langchain_core.messages import SystemMessage,  
HumanMessage, AIMessage
```

```
model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
```

```
system_msg = SystemMessage("You are a helpful assistant.")
```

```
human_msg = HumanMessage("Hello, how are you?")
```

Use with chat models

```
messages = [system_msg, human_msg]
```

```
response = model.invoke(messages) # Returns AIMessage
```

LangChain – ChatPromptTemplate



For dynamic chat messages, we use `chatPromptTemplate()`

the below supposed to work in langchain but unfortunately doesnot work for dynamic prompt

```
ChatPromptTemplate([
    (SystemMessage('You are a helpful {domain} expert')),
    (HumanMessage(content='Explain in simple terms, what is {topic}'))
])
```

- `SystemMessage()` and `HumanMessage()` are **already constructed message objects**
- They are not template-aware
- No variable injection happens

✓ **Use role + template string tuples works**

```
from langchain_core.prompts import ChatPromptTemplate

chat_template = ChatPromptTemplate([
    ("system", "You are a helpful {domain} expert."),
    ("human", "Explain in simple terms, what is {topic}.")
])

prompt = chat_template.invoke({
    "domain": "astronomy",
    "topic": "black holes"
})

print(prompt)
```



LangChain – ChatPromptTemplate

For dynamic chat messages, we use `chatPromptTemplate()` with chat history

```
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage
from langchain_google_genai import ChatGoogleGenerativeAI
from dotenv import load_dotenv
import os

# Load environment variables
load_dotenv()

# Initialize Gemini model
model = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash", # You can also use gemini-1.5-pro etc.
    temperature=0.7
)

# Initialize empty chat history
chat_history = []

# Create chat template with history placeholder
chat_template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful customer support assistant."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{query}")
])

print("Chat started. Type 'exit' to stop.\n")
```

```
while True:
    user_input = input("You: ")

    if user_input.lower() == "exit":
        print("Chat ended.")
        break

    # Create prompt including history
    prompt = chat_template.invoke({
        "chat_history": chat_history,
        "query": user_input
    })

    # Call Gemini
    response = model.invoke(prompt)

    # Print response
    print("AI:", response.content)

    # Update chat history
    chat_history.append(HumanMessage(content=user_input))
    chat_history.append(AIMessage(content=response.content))
```

Thank you